

모의고사 1회차 문제 1·2 풀이 정리

2026-10-09 · 문제 1: 군용 차량 수량 예측(YOLOv8) · 문제 2: 드론 비행 상태 분류 · 점수는 모두 공개 검증 기준이며 최종 test 점수가 아닙니다

1. 한눈에 보기

문제	과제 · 지표	최종 방법	검증 점수
문제 1	군용 차량 4종 수량 예측 이미지별 정확한 수량 일치율	yolov8n 사전학습 가중치를 학습 이미지 633장(라벨 점검 후)으로 100에폭 전이학습 → 좌우 반전 TTA와 박스 융합 → 클래스별 개수 세기, conf 0.4	평균 0.621 (conf 0.3~0.55 평균) 최고 0.631(@0.4, 낙관적) 정직한 추정 약 0.61 전부 0 예측 기준 0.381
문제 2	드론 비행 상태 5종 분류 Macro F1 (사용자 목표: 정확도 95%)	같은 비행(uid) 안의 과거 구간으로 만든 특징 59개 → 계층 분류(IDLE/HMSL/TURN/상승·하강 묶음 → 2단계) → RF-HGB 확률 평균 → ASC×3-DESC×5 가중	Macro F1 0.736 (가중 없음) 0.782 (가중, 낙관적) 정확도 0.840 · 95% 미달성

0.62

문제 1 검증 일치율(평균)

0.78

문제 2 Macro F1(낙관적, 가중 없음 0.74)

±2~4%p

문제 1 검증 노이즈(이미지 1장 = 0.6%p)

0.93

문제 2 경계 오류를 지워도 달는 정확도 상한(진단용)

세 가지 핵심 메시지

- 두 문제 모두 **모델 구조보다 데이터·라벨의 한계**가 점수를 가둡니다. 문제 1은 클래스 경계가 출처마다 다른 라벨과 작은 객체, 문제 2는 상태가 1~2행 만에 바뀌는 라벨 깜빡임입니다.
- 문제 1은 **에폭 증가와 좌우 반전 TTA**만 근거 있는 개선이었고(각 +0.02~0.03), 해상도·증강·손실 가중·2단계 구조 등은 모두 검증 노이즈 안이었습니다.
- 문제 2는 **긴 창(100~200행) 특징**이 가장 효과가 컸고(ASCEND F1 0.46 → 0.70), stack은 이득이 작은 데 비해 비용이 5배라 채택하지 않았습니다.

먼저 알려 둘 점

- 문제 1의 노트북과 제출 파일이 서로 다른 버전입니다.** 현재 submission.csv 는 병행 세션이 만든 v3(100에폭 + TTA + 박스 융합)이고, work.ipynb 는 이전 메인 v2(40에폭, conf 0.3)를 재현합니다. 노트북을 v3로 맞추는 작업은 아직 하지 않았습니다.
- 모든 점수는 공개 검증 데이터로 켜 값이고 conf·가중치를 같은 데이터로 골랐기 때문에 **낙관적**입니다. test 점수는 알 수 없습니다.
- 서버(CPU 2코어, Python 3.9, torch 1.10)에서의 실행 시간과 ultralytics 설치 여부는 측정하지 못했습니다.

2. 문제 1 — 군용 차량 수량 예측 (YOLOv8)

2.1 문제와 데이터

이미지에서 전차(tank), 군용 트럭(truck), 기타 군용 차량(military_vehicle), 민간 차량(civilian_vehicle)이 각각 몇 대인지 맞추는 문제입니다. 한 이미지에서 4개 수량이 **모두** 맞아야 1점이라 엄격합니다. 학습 640장, 검증 160장, 테스트 200장이며 모델은 YOLOv8 계열 사전학습 가중치(yolov8n)로 제한됩니다.

- 학습 이미지의 **48.8%(312/640)**는 대상 차량이 하나도 없습니다. 전부 0으로 예측해도 검증 일치율이 0.381이라, 이것이 비교 기준입니다.
- 클래스가 심하게 한쪽으로 쏠려 있습니다. 학습 이미지 수가 tank 248장인데 truck 53, military_vehicle 48, civilian 44장입니다.
- 박스가 작습니다. 학습 박스 중 이미지 면적의 1% 미만인 비율은 tank 34%, truck 39%, military_vehicle 53%, civilian 53%입니다.

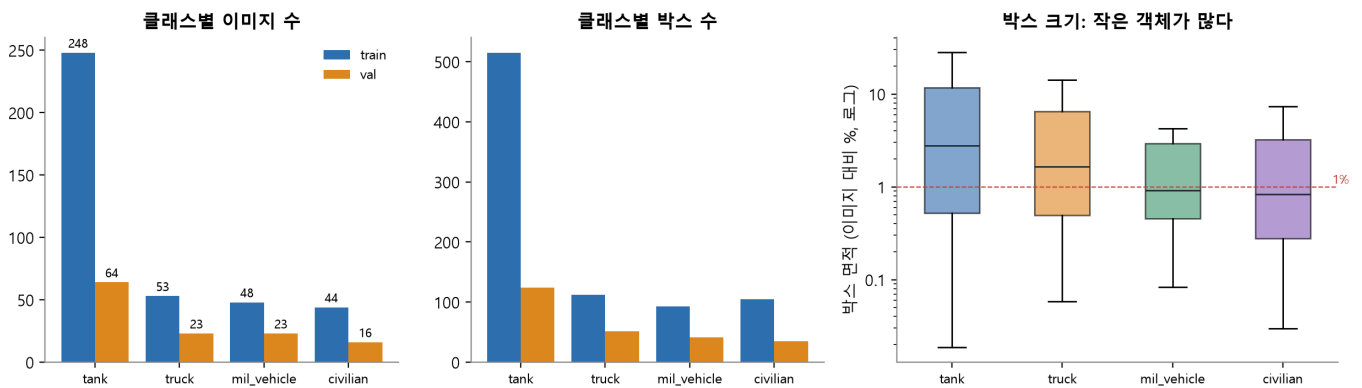


그림 1. 클래스별 이미지·박스 수와 박스 크기 분포(학습). 점선은 이미지 면적의 1%.

2.2 접근

사전학습 yolov8n을 학습 이미지로 전이학습하고, 테스트 이미지에서 검출된 박스를 클래스별로 세어 수량으로 제출합니다. 학습된 모델 자체보다 **후처리(신뢰도 임계값 conf, NMS, TTA)**가 일치율을 좌우하므로 검증 160장에서 conf를 0.15~0.6 범위로 탐색했습니다. 최종 후처리는 다음과 같습니다.

- 좌우 반전 이미지에서도 추론해 두 결과의 박스를 클래스와 무관하게 군집으로 융합(WBF)합니다.
- 융합된 박스 중 신뢰도 0.4 이상만 세어 클래스별 수량으로 만듭니다.

2.3 학습 곡선

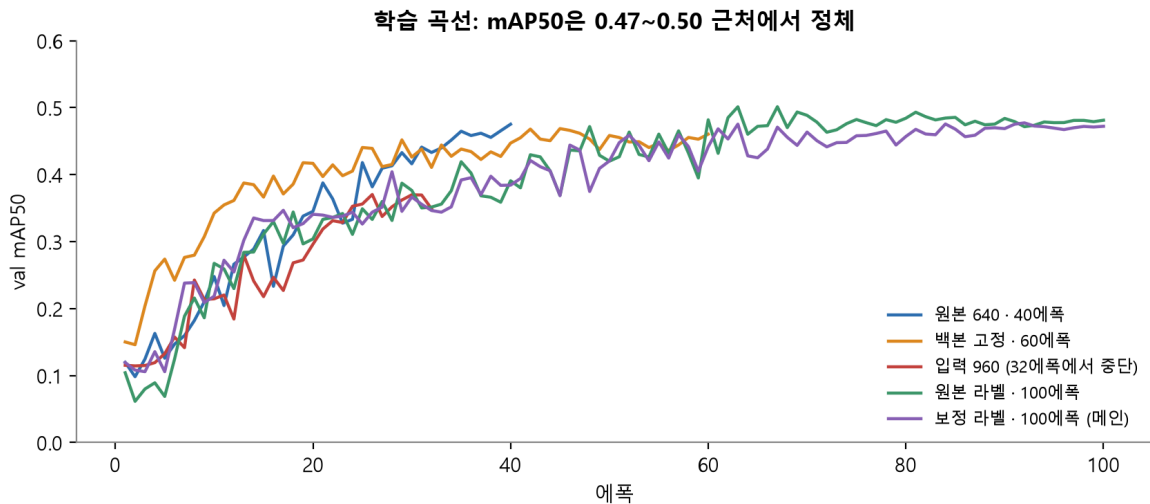


그림 2. 검증 mAP50 곡선. 설정이 달라도 0.47~0.50 근처에서 정체합니다. 입력 960 실험은 32에폭에서 중단돼 비교에서 제외했습니다.

2.4 실험 결과

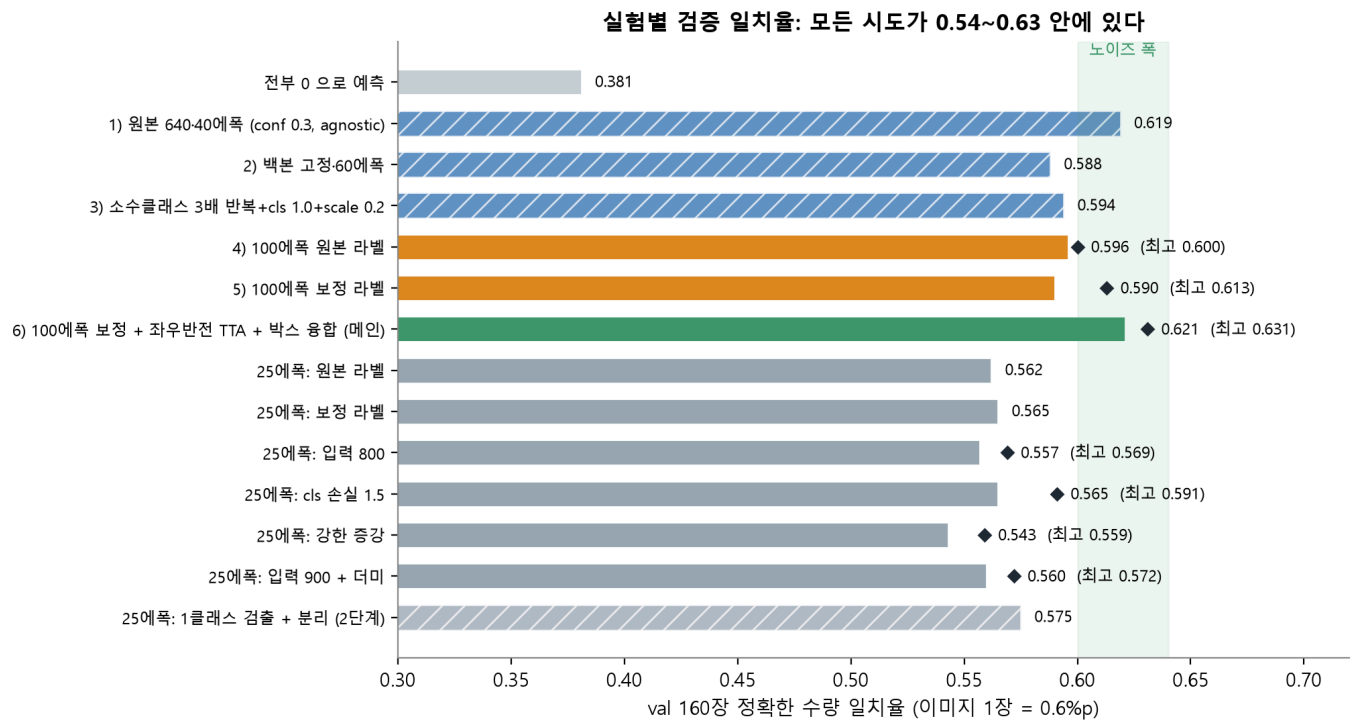


그림 3. 실험별 검증 정확한 수량 일치율. 막대는 conf 0.3~0.55 구간 평균, 사선 막대는 평균을 재지 않아 최고값(또는 선택값)만 표시한 것, ◆는 검증에서 고른 최고값(낙관적)입니다. 1~3번은 이 세션의 실험, 4~6번과 25에폭 묶음은 병행 세션의 기록입니다. 이미지 1장 = 0.6%p이고 시드만 바뀌어도 $\pm 2\%p$ 라, 0.60~0.64 범위의 차이는 구분되지 않습니다.

시도	판정	근거
에폭 25 → 100	효과 있음	약 +0.02~0.03 (0.565 → 0.58~0.60)
좌우 반전 TTA + 박스 융합	효과 있음	단일 모델 대비 약 +0.03, 임계값 민감도도 낮아짐
학습 입력 800-900-960, cls 손실 1.5, 강한 증강 (mixup·회전 등)	노이즈/악화	기준 0.565 대비 0.543~0.565
백본 고정(freeze 10) · 소수 클래스 3배 반복 + cls 1.0 + scale 0.2	기각	0.588 · 0.594 (기준 0.619보다 낮음)
축소·확대·회전 더미 데이터 583장 추가	노이즈	0.560 (mAP50은 0.47~0.51)
1클래스 검출 + 클래스 분리(2단계)	기각	1클래스 mAP50은 0.70이지만 총 대수 정확도 0.70으로 4클래스와 같음
6~7개 모델 융합, 클래스별 가중·임계값, 크기별 사전확률	노이즈	단일 100에폭 + TTA 이상 아님, 반 나눠 평가 시 -0.01~+0.01
라벨 점검(누락 박스 7개 추가, 삭제 2·재라벨 3·이미지 제외 7)	중립	원본 vs 보정 차이가 노이즈, 데이터는 보정판 유지

2.5 오류 구조: 어디서 틀리나

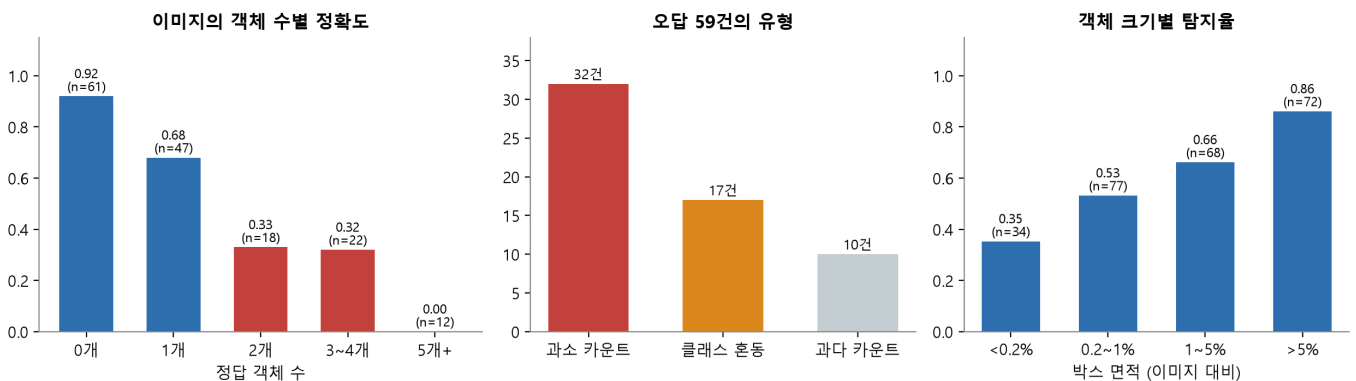


그림 4. 검증 오류 구조. 왼쪽·가운데는 100에폭 보정 모델 + TTA 기준, 오른쪽 객체 크기별 탐지율은 40에폭 원본 모델(conf 0.3) 기준입니다.

- **객체가 많은 이미지일수록 무너집니다.** 0개 이미지는 92%, 1개는 68%, 2개 이상은 32~33%이고, 5개 이상인 이미지 12장은 전부 틀립니다. 하나만 놓쳐도 0점이라 객체 수가 곧 위험입니다.
- **오답 59건 중 과소 카운트가 32건입니다.** 클래스 혼동이 17건, 과다 카운트가 10건입니다.
- **작은 객체일수록 못 잡습니다.** 면적 0.2% 미만 객체의 탐지율은 0.35, 5% 초과는 0.86입니다. 작은 객체는 보통 여러 대가 함께 있는 장면이라 두 오류가 겹칩니다.
- truck은 검증 정답 51개 중 12개만 예측됩니다. truck, military_vehicle, tank의 경계가 라벨 출처마다 달라 외형만으로는 구분이 안 되는 것이 천장 요인입니다. 총 대수 정확도 약 0.70 × 클래스 일치 약 0.85 ≈ 0.60이 대략의 상한입니다.

2.6 클래스 혼동과 분포 이동

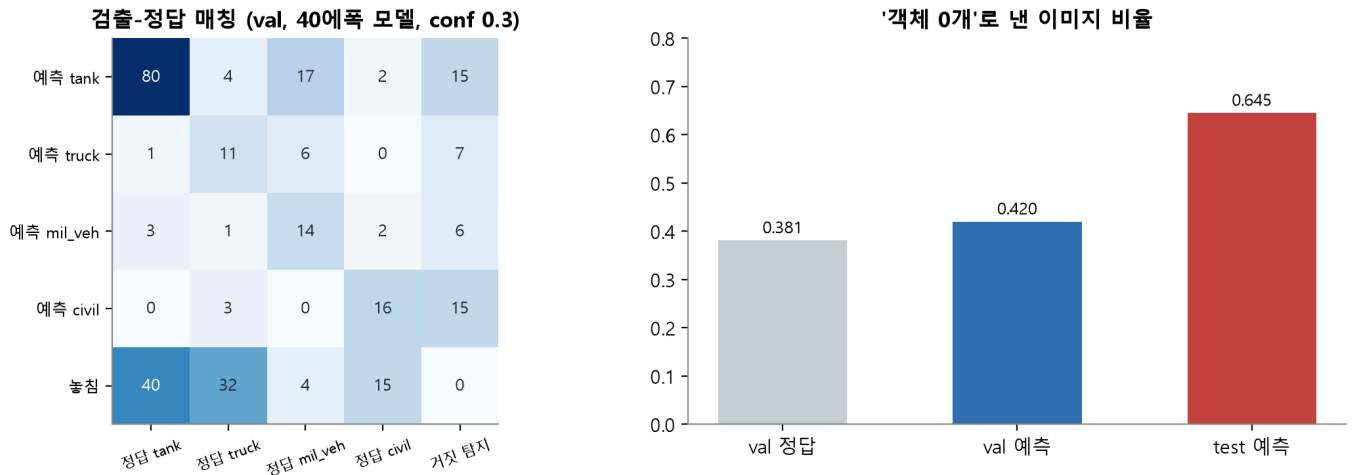


그림 5. 왼쪽: 검증 정답 박스와 예측 박스의 매칭(40에폭 모델, conf 0.3). 정답 251개 중 맞음 121, 클래스 혼동 39, 놓침 91이고 거짓 탐지가 43개입니다. 오른쪽: 객체가 없다고 낸 이미지 비율.

- 가장 큰 오류는 **놓침**(tank 40, truck 32)이고, 그다음이 military_vehicle를 tank로 부르는 혼동(17건)입니다.
- 테스트는 검증과 구성이 다를 수 있습니다.** 이미지 크기가 출처의 단서인데, 640×640 이미지는 학습+검증에서 82%가 빈 라벨이고 tank 외 클래스가 0건입니다. 이 크기가 테스트에는 47.5%(95/200)로 학습+검증(33%)보다 많습니다. 예측 기준 '객체 0개' 이미지 비율도 검증 0.42, 테스트 0.645로 크게 다릅니다.
- 테스트에서 0개로 예측한 이미지 중 신뢰도가 높은 것들을 눈으로 보면 헬기·항모·자주포 등 4종 밖 객체가 대부분이라 테스트에 빈 이미지가 더 많은 것으로 보이지만, 단정은 못 합니다. 따라서 **test 점수는 검증 점수와 다를 수 있습니다.**

2.7 최종 상태와 한계

항목	내용
메인 제출	<code>problem01/submission.csv</code> = <code>versions/v3_a100_fix_flipTTA_conf0.4.csv</code> (200행, 형식 검증 통과). 이전 메인(40에폭, conf 0.3 agnostic)은 <code>versions/_prev_main_backup_v2_gpu640.csv</code> 로 보존
검증 점수	평균 0.621, 최고 0.631(@0.4). 임계값을 반으로 나눠 튜닝해 다른 반에서 평가한 정직한 추정은 약 0.61
노트북	<code>work.ipynb</code> (10:51 저장)는 v2(40에폭, conf 0.3)를 재현하고 출력이 저장돼 있음. 현재 제출 v3와 다름. v3 반영은 미실시
서버 호환	미점검. 100에폭·TTA가 서버 CPU 2코어에서 몇 분인지, <code>ultralitics</code> 가 Python 3.9·torch 1.10에서 설치되는지 측정하지 못함
남은 선택지	(1) 정리 데이터로 100에폭 재학습, (2) train+val 합쳐 재학습(검증 불가), (3) 현재 유지

3. 문제 2 — 드론 비행 상태 분류

3.1 문제와 데이터

드론의 IMU(각속도·선형 가속도), 배터리, 바람 센서 시계열에서 각 관측 시점의 비행 상태(IDLE_HOVER, ASCEND, DESCEND, TURN, HMSL)를 맞추는 문제로, 지표는 Macro F1입니다. 사용자 목표는 **정확도 95% 이상**으로 정했습니다.

- 학습 41,669행(비행 5개: 002, 003, 005, 006, 007), 테스트 10,499행(비행 2개: 001, 004). 비행이 학습·테스트에 겹치지 않아 검증도 **비행 단위(Leave-One-Flight-Out)**로 합니다.
- 클래스 불균형이 큼니다. IDLE_HOVER 54.4%, HMSL 29.8%, TURN 9.6%, DESCEND 3.3%, ASCEND 2.9%입니다. 전부 IDLE로 찍어도 정확도 0.544입니다.
- DESCEND는 비행 002·003에 없고, TURN은 비행 007에 없습니다. 비행을 하나 빼면 폴드마다 클래스가 빠집니다.
- 학습 데이터의 상태 구간 4,985개 중 63%가 1~2행짜리입니다. 상태가 매우 자주 바뀌는 라벨입니다.

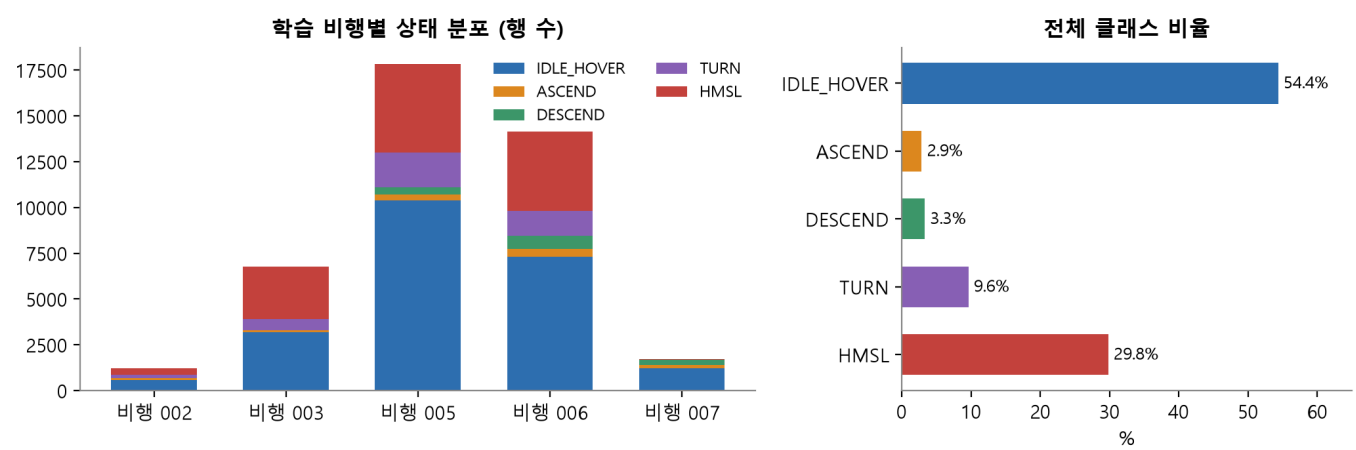


그림 6. 학습 비행별 상태 분포(왼쪽)와 전체 클래스 비율(오른쪽).

3.2 데이터 분석: 어떤 값이 상태를 가르나

센서 10개와 과거 구간 파생 특징 약 60개를 비행마다 중앙값을 빼서(비행 간 이동 보정) 상태 구분력을 비교했습니다. η^2 는 상태에 따라 값이 얼마나 다른지, AUC는 한 상태를 나머지와 구분하는 정도(0.5 = 구분 없음)입니다.

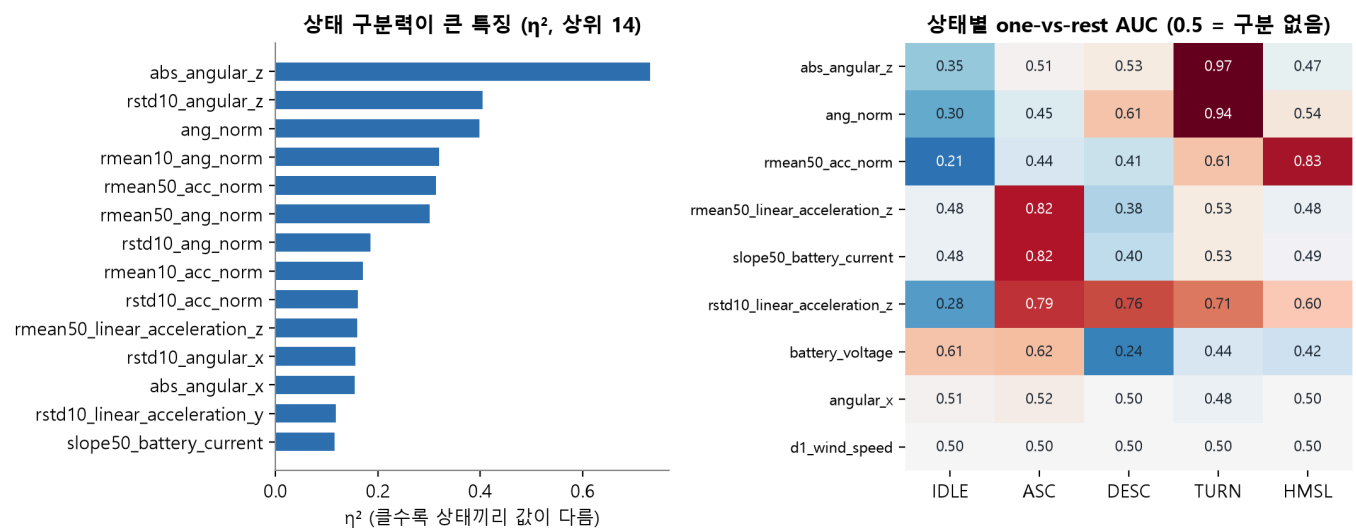


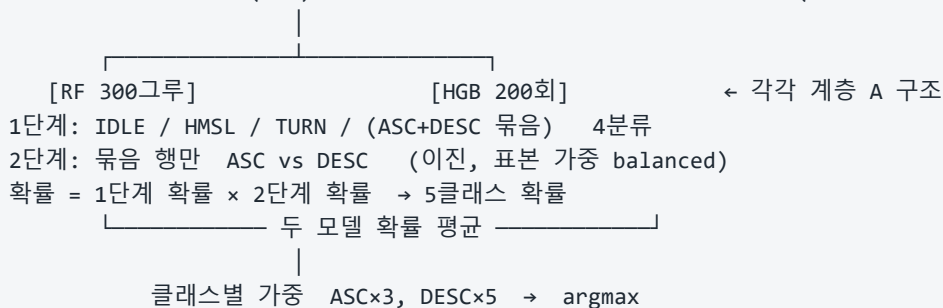
그림 7. 왼쪽: 구분력이 큰 특징. 오른쪽: 대표 특징의 상태별 AUC. 파랑은 0.5보다 낮은 방향, 빨강은 높은 방향입니다. 이 분석은 학습 데이터 안에서만 했고 테스트 정답은 쓰지 않았습니다.

상태	특이한 값 (AUC)	해석
TURN	각속도 z 절댓값 0.97, 각속도 크기 0.94	가장 쉬운 상태. 각속도 크기 하나로 거의 걸립니다.
IDLE ↔ HMSL	50행 평균 가속도 크기: IDLE 0.21, HMSL 0.83	움직임의 크기 축에서 반대편에 있습니다.
ASCEND	z축 가속도 평균 0.82, 배터리 전류 기울기 0.82	z축 가속도가 +쪽이고 전류가 증가합니다.
DESCEND	z축 가속도 변동 0.76, 전압 0.24	어떤 값으로도 AUC가 0.76을 넘지 못해 가장 어렵습니다.
구분 안 되는 값	부호 있는 x·y 가속도, x축 각속도, 1행 변화량 (d1_*), payload	부호가 상쇄되거나 센서가 정수로 양자화돼 신호가 없습니다. 절댓값이나 표준편차로 바꿔야 쓸모가 생깁니다.

- ASCEND와 DESCEND를 서로 직접 비교하면 z축 가속도 평균 부호가 같았습니다(ASCEND +0.099, DESCEND -0.068, 비행 3개에서 방향이 같음). 반면 배터리 전압 기울기는 ASCEND만 가르고 DESCEND는 가르지 못했습니다.
- 테스트 비행은 센서 분포가 학습과 다릅니다. 풍속 평균 6.34 → 3.79, 배터리 전압 평균 45,970 → 47,348입니다. 그래서 **절댓값 센서는 쓰지 않고 변화량·비율·구간 통계만** 씁니다.

3.3 모델 구조

센서 10열 → 같은 비행(uid) 안의 현재+과거 행으로 특징 59개 생성 (미래 행 미사용)



특징 그룹 (59개)	내용
각속도·가속도 크기 (8)	축별 절댓값, 3축 벡터 크기
구간 평균 (15+3)	창 5·10·50·100·200행의 각속도 크기·가속도 크기·부호 있는 z가속도 평균, 각속도 z 절댓값 평균
변동·극값 (12+3)	창 10·50행의 표준편차·최대·최소, 축별 변동
지수이동평균 (9)	span 5·20·100의 가속도 크기·각속도 크기·z가속도
배터리 (7)	전류 기울기(10·50·100행), 전압 기울기(50·100행), 비행 시작 대비 전압 변화, 전류 50행 평균
추세 (2)	10행 평균 - 50행 평균 (가속도, 각속도)

3.4 실험 결과

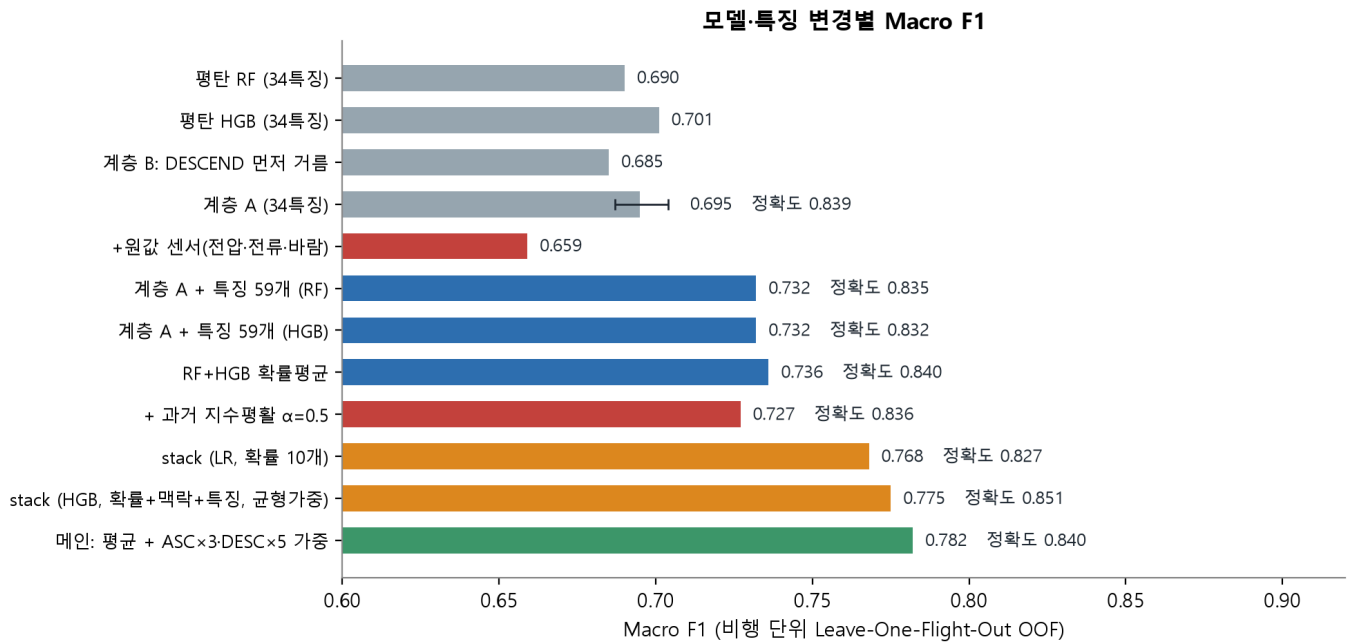


그림 8. 변경별 Macro F1(비행 단위 Leave-One-Flight-Out OOF). 파랑은 채택 경로, 주황은 stack 실험, 빨강은 기각입니다. 가중치(ASC×3, DESC×5)는 OOF 점수로 골라 낙관적이고, 계층 A(34특징)는 두 번 측정된 값(0.687, 0.704)의 폭을 막대로 표시했습니다. 폴드가 5개이고 DESCEND 구간이 31개뿐이라 0.01 안팎의 차이는 노이즈일 수 있습니다.

- **특징 확장이 가장 효과가 컸습니다.** 100~200행 창과 지수이동평균을 더하자 Macro F1이 0.69에서 0.73으로, ASCEND F1이 0.46에서 0.70으로 올랐습니다. 정확도는 거의 그대로였습니다.
- **원값 센서(전압·전류·바람)를 넣으면 크게 떨어집니다**(0.69 → 0.64~0.66). 비행 간 분포 이동을 타는 값이라 제외했습니다.
- **DESCEND를 먼저 전용 모델로 거르는 구조(B)는 오히려 낮았습니다**(0.685). 임계값을 올리면 DESCEND F1이 0.17까지 떨어집니다. 거짓 양성이 많아집니다.
- **과거 지수평활은 해로웠습니다.** 구간이 짧아 전환을 뭉개 TURN이 크게 떨어집니다($\alpha=0.8$ 이면 TURN F1 0.94 → 0.75).
- **stack은 이득이 작고 비용이 큼니다.** 공정하게 같은 조건(사후 가중 없음)으로 비교하면 정확도 0.840 → 0.851, Macro F1 0.736 → 0.775였지만, 낙관적 가중 버전끼리는 0.782 대 0.786으로 거의 같습니다. stack 1 생성이 개발 PC에서 1,186초라 서버 2코어에서는 25~40분(추정)이 걸려 채택하지 않았습니다. HGB만으로 줄여 다시 시험했지만 비행 하나에 233초(RF 포함 270초)로 절감이 거의 없어 중단했습니다.

3.5 최종 성능

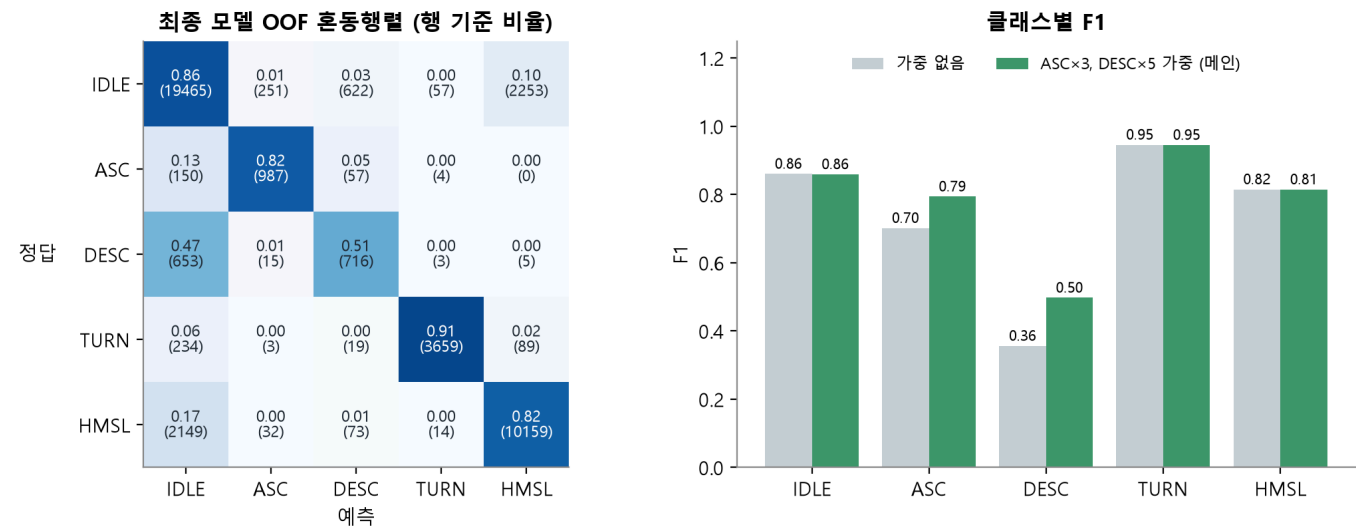


그림 9. 최종 모델의 OOF 혼동행렬(왼쪽, 가중 적용)과 클래스별 F1(오른쪽). 학습 비행 5개를 하나씩 빼고 만든 OOF 예측이며 정확도 0.840(가중 없음) / 0.840(가중), Macro F1 0.736(가중 없음) / 0.782(가중)입니다.

- TURN(F1 0.95), IDLE(0.86), HMSL(0.81)은 안정적입니다. 가중은 이 셋을 거의 건드리지 않고 ASCEND(0.70 → 0.79)와 DESCEND(0.36 → 0.50)만 올립니다.
- 가장 큰 오류 두 가지: IDLE과 HMSL이 서로 섞이는 것(IDLE → HMSL 2,253행, HMSL → IDLE 2,149행)과, DESCEND의 47%가 IDLE로 빠지는 것입니다.

3.6 정확도 95%에 닿지 못한 이유

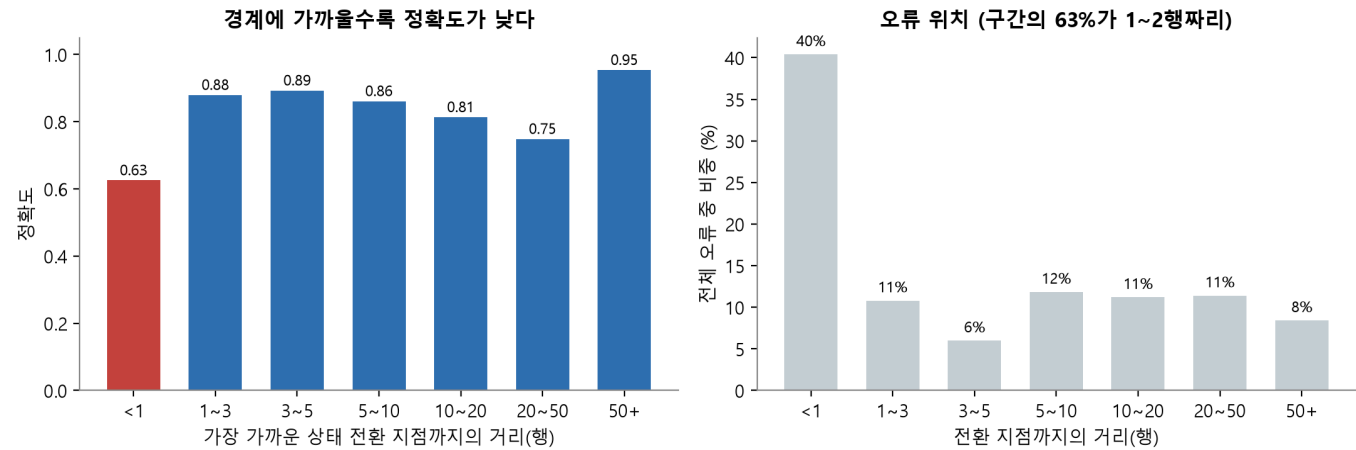


그림 10. 가장 가까운 상태 전환 지점까지의 거리별 정확도(왼쪽)와 오류의 위치 분포(오른쪽). 가중 없음 기준 OOF입니다. '<1'은 전환 직전·직후 행입니다.

- 오류의 57%가 상태 전환 지점 5행 이내에서 나고, 전환 바로 옆 행의 정확도는 0.63입니다. 구간의 63%가 1~2행이라, 센서만 보고는 알 수 없는 라벨 깜빡임에 가깝습니다.
- 전환 5행 이내의 오류를 전부 없앴다고 가정해도 정확도 상한은 0.931입니다. 95%에 닿으려면 전환에서 먼 구간의 오류까지 줄여야 하는데, 전환에서 20~50행 떨어진 구간의 정확도도 0.75로 낮습니다.
- 따라서 이번 시도로는 정확도 95%를 달성하지 못했고(0.840), 현재 접근으로 닿을 가능성도 낮다고 판단합니다. 개선은 Macro F1에서만 확인됐습니다.

3.7 최종 상태와 한계

항목	내용
메인 제출	<code>problem02/submission.csv</code> = <code>versions/v2_featx_ens_wt_main.csv</code> (10,499행, <code>id,flight_state</code> , 형식·ID 순서 검증 통과)
노트북	<code>work.ipynb</code> Restart & Run All로 출력 저장, 오류 없음. 현재 제출과 일치
점수의 신뢰도	가중치(ASC×3, DESC×5)는 OOF로 골라 낙관적. 비행이 5개뿐이고 DESCEND 구간이 31개라 변동이 큼. 보수적인 값은 가중 없음(Macro F1 0.736)
테스트 예측 분포	IDLE 52.6%, HMSL 29.6%, TURN 13.0%, ASCEND 3.4%, DESCEND 1.5% (학습은 54.4 / 29.8 / 9.6 / 2.9 / 3.3). TURN 비중 차이의 원인은 확인하지 못함
서버 호환	HGB의 <code>class_weight</code> (scikit-learn 1.2+)를 <code>sample_weight</code> 로 교체해 1.1.3 호환으로 수정(예측 100% 동일). 서버에서의 실행은 미확인. 서버 2코어 소요 시간은 개발 PC(약 2분)의 4~8배로 추정(미측정)
남은 선택지	IDLE ↔ HMSL 개선용 긴 구간·주파수 특징(기대 폭 작음)

4. 정리: 두 문제에서 배운 점

공통 교훈	내용
천장 진단을 먼저	개선에 들어가기 전에 오류가 어디서 나는지(객체 수, 경계 근처, 클래스 혼동)부터 보면, 구조 변경이 효과 없는 이유와 현실적인 상한이 먼저 보입니다.
검증 노이즈 범위를 의식	문제 1은 이미지 1장이 0.6%p, 문제 2는 DESCEND 구간이 31개입니다. 이 폭 안의 차이로 모델을 고르지 않습니다.
분포 이동에 둔감한 특징	문제 2의 원값 센서처럼 비행·출처에 따라 달라지는 값은 검증에서는 좋아 보여도 일반화에 해롭습니다.
비용 대비 이득	<code>stack</code> 과 960 입력처럼 서버에서 비싼 방법은 이득이 작으면 채택하지 않습니다(서버는 CPU 2코어, 개발 PC 대비 4~8배 느릴 것으로 추정).
제출·노트북 일치	문제 1처럼 병행 작업으로 제출 파일만 바뀌면 노트북과 어긋납니다. 마감 전에 두 파일을 같이 확인해야 합니다.

산출물 위치: `training_s/26_10_09/all_data/problem01` , `problem02` 의 `work.ipynb` , `submission.csv` , `versions/EXPERIMENTS.md` . 문제 2 실험 스크립트는 `problem02/versions/scripts/` 에 있습니다.